Forge

An engine, harness and operating layer where every app is its own world, every world is a self-hosted node, and worlds link to each other across the internet.

After the abstract, this document has three parts:

- **1. Mission statement**: what Forge is for and the principles it will not trade away.
- **2. Specification**: the architecture, the formats and protocols, and the exact state of every piece (built and tested, scaffolded, or planned).
- **3. Wants and needs for the next model**: the ordered work list with what is now done, acceptance tests, the guardrails, the open decisions, and what the human owner has to provide.

Status at this handoff (third edition, October 2026): 107 automated tests pass in the TypeScript engine and 6 in the Rust shim. Static Linux programs for x86-64 and ARM64 run inside Forge and print exactly what real hardware prints, now including ordinary statically linked glibc programs (printf, malloc, floating point) on x86-64. Forge ships as a command-line program and as a single-file Linux executable built offline. The desktop app is fully configured but has never been compiled, because this build environment could not reach crates.io or npm.

Abstract

Forge: an AI-native multiverse engine

Forge is a cross-platform game and application engine in which every instance is a self-hosted world, and worlds link into a shared multiverse. Each world runs on its owner's hardware, x86 or ARM, desktop or mobile, and joins the network through signed portals; when its owner stops it, it leaves.

Contributors share skills, mechanics and code. An in-engine agent decompiles each contribution into a neutral intermediate form, rewrites it, verifies it against a test fixture, and lowers it to WebAssembly. The result is one optimized, portable form that any instance can run, whatever its screen or processor. As of this edition that path is real for ordinary compiled programs: x86-64 and ARM64 Linux executables are lifted into one IR and run as WebAssembly with results identical to the processor they were built for. Contributions can be fused: one world's outputs wired to another's inputs, producing mechanics no single author wrote.

A small local model routes user intent to verified skills. A frontier model authors new skills, which become permanent local features once they pass their tests.

Because compute varies across devices, rendering adapts continuously. A Nanite-style cluster level-of-detail system acts as the master throttle, scaling geometry, resolution, Gaussian splats and neural textures to each device's budget. Every participant sees the same multiverse at the fidelity their hardware can sustain.

Forge runs two ways, and permanence is always the owner's choice. By default it is a normal app that runs inside any existing operating system on any hardware. For those who want the most from it, Forge can replace the original operating system entirely and boot as the system itself. Either way, everything routes through it and the device's own functions (calls, text messages, notifications, files, other apps) are passed through, so a phone stays a fully working phone and a PC stays a fully working PC. Each person's setup is shaped by their own taste.

One Forge spans all of a person's devices. Rather than a separate install on each, phones, PCs, TVs and tablets link into a single shared instance with the same worlds, skills and saves. Unlike casting, every linked device can both display and control, each renders at its own resolution, and the simulation itself moves to whichever device is strongest at the moment.

That same pooling makes Forge one AI computer. Linked devices split a model between them, so an owner can run models larger than any single device could hold, natively, with the engine's skills and worlds on top. The short version: your devices become one AI computer, and one world.

1. Mission statement

> *Every app is a world. Every world is a node someone owns. Worlds connect like web pages do, and a world lives exactly as long as its owner keeps it running.*

Forge is a single program that runs the same way on x86 and ARM, on Windows, macOS, Linux and mobile. It opens its own window like any desktop app. Inside it, each application runs as an island: its own functional universe with its own rules, scene and tools. Islands are hosted by the people who make them, on their own machines, and they link to each other through portals. Standing next to another person's island, or stepping through a portal into it, should feel like moving through one continuous place. When an owner shuts their app down, their patch of the network disappears with it.

What it is for

- **A universal runtime.** Software arrives in many forms (WebAssembly, scripts, 3D assets, models, native binaries). Forge identifies each, routes it to the right layer, and runs what it can verify.
- **One portable bytecode.** WebAssembly is the common layer. Code written for x86 or ARM is lifted into a neutral form and lowered to WebAssembly, so it runs anywhere Forge runs.
- **Intent in, features out.** A small local model is a router of intent. The skills and tools are the engine features. A frontier cloud model writes new skills; once a skill passes its test, it is local forever and no longer needs the cloud.
- **Fusion.** Apps combine by wiring one island's outputs to another's inputs, not by mashing code together.
- **Guest or native, your choice.** Forge is a complete, normal app inside any operating system. Owners who want the most from it can drop their original operating system and boot Forge instead. The same engine, worlds and skills work in both modes; phone and PC functions are passed through, never lost.
- **One Forge, every device.** A person's devices link into one instance instead of each running its own. Any device can show, control, compute or host, and the work moves to wherever it runs best.
- **One AI computer.** Linked devices pool memory and compute to run local models bigger than any one of them could. This is the first reason AI-focused users install Forge, before the multiverse exists.

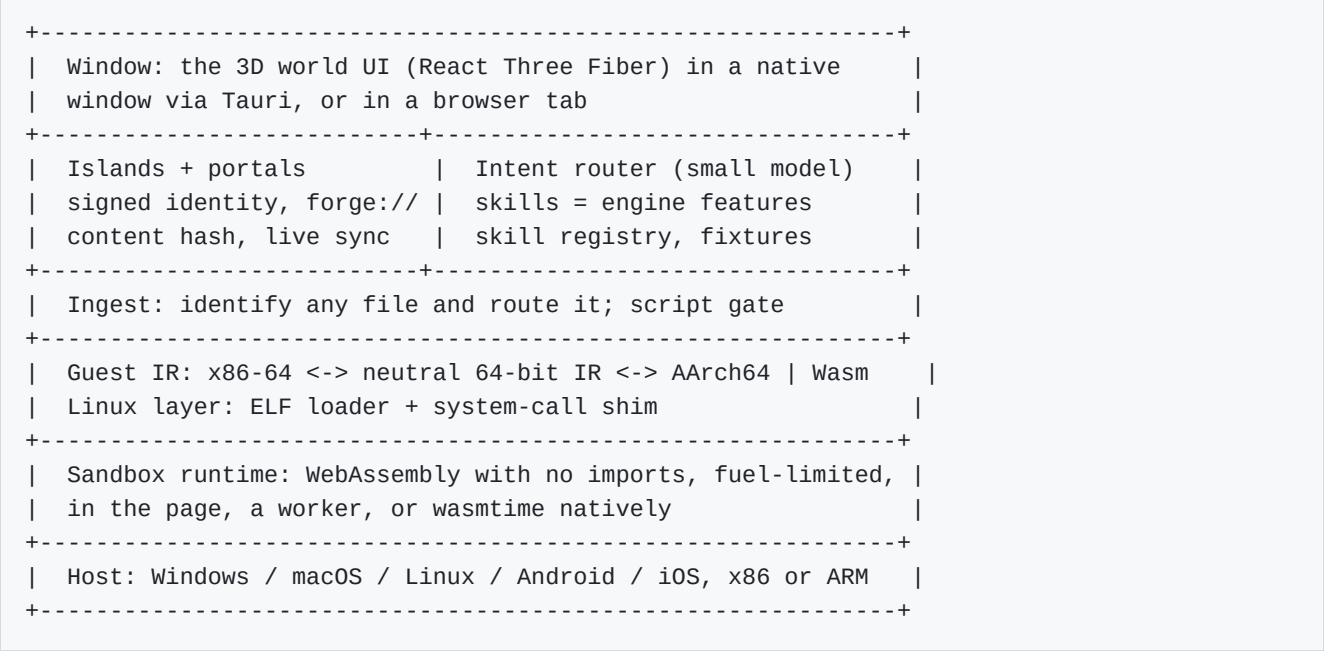
Principles

- **Owners own their nodes.** No central server is required to host or reach an island.
- **Identity is a key.** An island is whoever holds its private key. Nothing can claim an island's name without it.
- **A request is never a grant.** Visitors ask; hosts decide. Every capability is explicit and minimal.
- **Unknown code does not run.** Bytes the engine cannot verify are reported as gaps, never executed.

- **Verified before trusted.** Every new translation rule, skill or injected module must pass a fixture test before the system uses it.
- **Permanence is optional.** Running as an app is the default and is never a lesser version. Taking over a device is opt-in, and the owner can return to the stock system at any time without losing data.
- **Honest about limits.** Some software cannot be run or translated perfectly. Forge says so rather than pretending.

2. Specification

2.1 Layers



2.2 Component status

Component	Files	State
Movement and cameras (third person, first person, bench)	components/forge/pilot.tsx, look.tsx	Fixed in code. Not yet tested by hand in a browser. Untouched this pass.
Avatar recovery	engine/manifold.ts (ensureAvatar), store.ts	Built, tested
Renderer quality by device	engine/hardware.ts, Viewport.tsx	Built, tested. Software overlay off unless ?soft=1
File ingest	engine/ingest.ts, components/forge/Bridge.tsx	Built, tested. Static Linux programs are now "ready". Drop-a-file UI added (not hand-tested).
Guest IR v2 (x86-64 and AArch64 in, WebAssembly out)	engine/guest/ir.ts, x86.ts, arm.ts, lift.ts, machine.ts, wasm.ts	**New. Built, tested against the real CPU**: 8 compiled builds of one C program (gcc and clang, -O0 to -O3, both architectures) and 150 random programs, each 4 ways
Linux layer	engine/guest/elf.ts, linux.ts, linux-worker.ts, linux-client.ts; scripts/forge-run.ts	**New. Built, tested.** Static, non-TLS programs (freestanding or musl-style)

First compat IR (four ops)	engine/isa.ts, wasm-backend.ts	Kept for the in-world harness demo and its tests; superseded by guest/
Native-or-browser run call	engine/native-host.ts	Built, tested (first IR only)
Island identity, manifest, portal handshake	engine/island.ts	Built, tested, including attack cases
Self-hosted island node	scripts/island-node.ts	Built, tested over real HTTP. Now serves its world and live sync
Island content and live sync	engine/island-content.ts, portal-client.ts (enterIsland, syncIsland)	**New. Built, tested** with two visitors over HTTP on one machine
Portals in the world	island-content.ts (makePortal, portalAt), portal-tool.ts, Bridge.tsx, Viewport.tsx	**New.** Engine built and tested; UI wired, not hand-tested
Skill registry and router	engine/skills.ts, bounded.ts	**New. Built, tested** (wasm and ARM-Linux skills, offline routing)
Safe-compile gate for scripts	engine/script-gate.ts	**New. Built, tested** (static layer)
Pooled model inference	engine/pooled.ts, scripts/compute-node.ts	**New. Built, tested** across separate processes with a toy model
Desktop host (Tauri + wasmtime)	src-tauri/	Written. Icons, static frontend build and installer config in place. Never compiled
Native guest runner (N1)	src-tauri/src/guest.rs, lib.rs native_guest; engine/native-host.ts runLinuxHosted	**New.** Shim and run loop tested in Rust (std only). Page protocol tested against a stand-in. wasmtime glue checked against API stubs only
glibc on x86-64 (N2)	engine/guest/x86.ts, ir.ts, machine.ts, wasm.ts	**New. Built, tested** against the CPU: 300 committed random programs (about 2,400 more run) plus glibc printf/malloc/float programs
Isolated script runtime (N3)	engine/script-runtime.ts	**New. Built, tested** (worker, message-only API, time limit)
Command line and standalone executable	scripts/forge-cli.ts, build-cli.mjs, sea-inject.mjs, make-sea.sh; bin/	**New. Built and run here** (Linux x64 single file, no Node needed)
Multi-device linking	engine/device-link.ts	Built, tested. No transport or state streaming yet
Multiplayer replication	engine/replication.ts	Built, tested. Now wired to HTTP sync. No client-side prediction

Adjacency, native mode, device shell levels, local mesh, zones across machines	-	Planned

2.3 Island address

`forge://host[:port]/slug`. The host is a DNS name or IP, lowercased on normalization. The slug is lowercase letters, digits and hyphens, at most 64 characters. Anything else is rejected. Today the address maps to `http(s)://host:port` for transport.

2.4 Island manifest (version 1)

```
{
  "forge": "island",       "version": 1,
  "id":        "<first 22 chars of base64url(SHA-256(publicKey))>",
  "name":      "My Island",                (1 to 80 chars)
  "address":   "forge://example.com:7420/my-island",
  "publicKey": "<base64url raw Ed25519 public key, 32 bytes>",
  "entry":     { "kind": "wasm" | "world", "sha256": "...", "bytes": 0 },
  "wants":     ["render", "input"],          (capabilities this island asks for)
  "portals":   ["forge://other.example/their-island"]
}
signature = Ed25519( "manifest|" + canonical_json(manifest) )
canonical_json = JSON with object keys sorted, undefined dropped
```

New this edition: for a `"world"` entry, `sha256` and `bytes` describe the island's starting world, encoded as canonical JSON `{ "forge": "island-world", "version": 1, "entities": [...] }`. The owner's avatar is not part of the entry.

2.5 Capabilities

render, input, audio, storage, portal, fetch, compute. A host's policy lists what it allows; a grant is the intersection of what the visitor asked for and what the host allows. A host may also block island ids. The script gate uses the same names: a script may only touch fetch, storage, compute, audio or render APIs if its island was granted them.

2.6 Portal handshake

- **Visitor to host:** `hello` = { from: signed manifest, to: address, wants, nonce (16 random bytes) } signed over `hello|nonce|to|visitorId|sortedWants`.
- **Host checks:** the visitor manifest verifies; the id matches the key; the hello is addressed to this island; the hello signature verifies; the visitor is not blocked. Then it grants wants and allow.
- **Host to visitor:** `welcome` = { host: signed manifest, grant, nonce } signed over `welcome|nonce|visitorId|hostId|sortedGrant`. New: the HTTP reply also carries `session`, a random 144-bit bearer token for the content and sync endpoints (not part of the signature; use HTTPS across the internet).
- **Visitor checks:** the host manifest verifies; its address is the one asked for; the nonce matches; the grant is a subset of the request; the signature verifies.

- **Then:** the visitor downloads the entry and refuses it unless its size and SHA-256 equal the manifest's. Entities are structurally validated, placed beside the visitor's own island and tagged read-only.

2.7 Node endpoints

Method and path	Purpose
GET /.well-known/forge-island	The signed manifest
POST /portal/hello	Handshake. 200 with welcome and session, 403 refused, 400 malformed or over 64 KB
GET /island/content	The starting world. Needs `Authorization: Forge <session>` and the render grant
POST /island/sync	`{ view: { position, forward, fovDeg?, range?, tier?, budget? } }` → `{ tick, enter, update, leave, bytes, dropped }`. Only what this visitor can see, inside its byte budget, only what changed. With the input grant, the visitor appears to others as a host-owned presence marker
GET /health	Liveness. Offline is simply: this does not answer

Run a node: `node --experimental-strip-types scripts/island-node.ts --name "My Island" --slug my-island --port 7420 --allow render,input,portal [--world saved-world.json] [--tick 100]`. The key is stored in `.forge/identity.json` with owner-only permissions and survives restarts.

2.8 Guest IR and the Linux layer (new)

The first compat IR (four ops, 32-bit) is kept for the in-world demo. Real programs now go through `src/engine/guest/`:

```
static ELF (x86-64 or AArch64)
  -> lift: decode from the entry point, follow branches, calls and return sites
  -> guest IR (64-bit; widths 8/16/32/64 with x86 and ARM write rules)
  -> WebAssembly: one module, no imports, exports run(): i32 and its memory
  -> run until a system call -> Linux shim answers -> run again
  (the reference interpreter runs the same IR, exactly, for checking)
```

- **IR operations:** mov; add, sub, and, or, xor, shl, shr, sar, ror, mul, umulh, smulh, udiv, sdiv, urem, srem; sign/zero extension; lea; loads and stores of 1, 2, 4, 8 bytes; set, csel, ccmp; conditional branch, cbz, tbz, direct and indirect jump; syscall; guards for division faults; halt. Flags are Z, S, C (x86 carry/borrow), O, plus K (last flag-setter was a subtraction), so ARM's carry is C xor K.
- **x86-64 coverage:** prefixes 66, F3, REX; ModRM, SIB, RIP-relative; the ALU group (not adc/sbb); test, cmp, inc, dec, neg, not; shifts; imul (all forms), mul, div, idiv; mov, movzx, movsx, movsxd, lea, xchg; push, pop, call, ret, leave, jmp, jcc; setcc, cmovcc; cdq, cqo, cdqe; rep movs and rep stos; syscall; nop forms, endbr64.
- **AArch64 coverage:** movz, movn, movk; add/sub immediate, shifted and extended (with flags); logical immediate and shifted (including bic, orn, eon); ubfm, sbfm, bfm (lsl, lsr, asr, uxt*, sxt*, ubfx, bfi); extr as ror; madd, msub, smaddl, umaddl, smulh, umulh; udiv, sdiv; variable shifts; csel, csinc, csinv, csneg; ccmp, ccmn; b, bl, b.cond, cbz, cbnz, tbz, tbnz, br, blr, ret; ldr/str (unsigned offset, unscaled, pre/post-index, register offset, literal), ldrs*, ldp/stp, ldpsw; adr, adrp; svc; hints and barriers.
- **Safety:** unknown bytes are gaps and are never executed; the run stops at a gap with its address and bytes. Every memory access is bounds-checked; the first 4 KiB are a null page. A store into translated code stops the program ("self-modifying code is not supported"). Fuel bounds every run. The WebAssembly module imports nothing.
- **Linux shim:** write, writev, read (from a provided stdin), brk, anonymous mmap, munmap, exit, exit_group, uname, getpid, getuid, set_tid_address, clock_gettime (a fixed clock), getrandom, ioctl (ENOTTY), signal calls (accepted, ignored). Everything else returns -ENOSYS and is listed in the report. No files, sockets or processes are reachable.
- **How it is tested:** the real processor is the oracle. `fixtures/linux/` holds one C program (sieve, recursion, sort, hashing, a switch table, function pointers, signed and unsigned division, stdin) built 8 ways; every build must print exactly what the x86-64 build prints on real hardware, on both engines. `fixtures/compat-corpus.json` holds 150 random programs written as both x86-64 and AArch64 assembly and assembled by the real toolchains; each must give the hardware's answer 4 ways (x86 and ARM, interpreter and WebAssembly). 500 more were run during development. On its first run the corpus caught a real bug in the reference interpreter.
- **Speed (this machine):** the compiled workload runs in 0.1 to 0.3 s through WebAssembly versus 0.5 to 1.5 s in the interpreter.

- **Added in the third edition (x86-64):** fs: thread-local storage, cpuid (a baseline x86-64 with SSE2 and no AVX, vendor "ForgeVirtCPU"), SSE2 integer and data movement (lowered to WebAssembly SIMD), scalar SSE float with x86 NaN rules, parity, adc/sbb, rotates, shld/shrd, the bit-test family, bsf/bsr/tzcnt/lzcnt, cmpxchg/xadd, bswap, ah/bh/ch/dh, 128-by-64 division, carry/overflow after shifts, rotates and multiplies, the 67 prefix, rdssp, fnstcw/fldcw. Execution is tiered (interpreter, then WebAssembly), and programs are lowered as one function per 16 KiB of code.
- **Not yet:** AVX, x87 arithmetic, packed float SSE, VEX (BMI); ARM SIMD/FP, atomics, system registers (so ARM glibc stops at its first gap); dynamic linking; threads; files.

2.9 Ingest routes

Detected by	Kind	Route
00 61 73 6d	wasm	sandbox-vm (validated, imports and exports listed, not run)
GGUF	gguf	resident-model
glTF / .gltf	glb / gltf	scene-asset
PNG, JPEG, WEBP, GIF	image	texture
.js .ts .tsx .jsx .mjs .html	script	safe-compile gate (built)
PK.. or gzip	archive	unpack (not yet built)
ELF (static, x86-64 or ARM64)	native	Linux layer (built). Dynamic or PIE: refused with the reason
MZ (PE), Mach-O	native	reported, not run (no Windows or macOS layer yet)
anything else	unknown	reject

2.10 Skills (new)

```
{
  "name": "factorial", "version": 1,
  "description": "Factorial of a whole number n (n!).",
  "examples": ["what is 5 factorial"],
  "schema": { "type": "object", "properties": { "n": { "type": "integer", "minimum": 0,
"maximum": 20 } }, "required": ["n"] },
  "impl": { "kind": "wasm", "module": "<base64url>", "export": "fact" }      or { "kind":
"linux", "elf": "<base64url>" },
  "fixtures": [ { "args": { "n": 5 }, "expect": 120 } ]
}
```

- **Getting in:** `proposeSkill` checks the shape and schema, refuses modules with imports, then runs every fixture here; one mismatch and nothing is installed. Versions only move up. The registry stores a hash of each implementation and drops any skill whose bytes changed when it is loaded again.
- **Running:** wasm skills take the schema's numbers in order and return one number; they run in a worker that is terminated at the time limit. Linux skills (either architecture) get their arguments as canonical JSON on stdin and answer with JSON on stdout, through the Linux layer. Neither can reach the network.
- **Routing:** a picker (the resident small model, or the built-in word-overlap picker) returns `{ skill, args }`. The router never trusts it: an unknown skill is refused and arguments are schema-checked before anything runs. Tested with the network disabled.

2.11 Script gate (new)

Second layer (third edition): `runGatedScript(source, name, api)` gates the script, then runs it in its own worker where network, storage, workers, eval and the Function constructors have been removed and the shared prototypes frozen. The script's default export receives `forge`, whose methods only send messages; the host answers the ones it offers. A time limit and a call limit stop runaway scripts.

`gateScript(source, fileName, { grants, allowImports, typecheck })` uses the TypeScript compiler. It reports syntax errors, optional type errors (ES2022 library only, so host APIs are not even typed), and refuses: network, storage, WebAssembly, audio and canvas APIs unless granted; and always eval, the Function constructor, globalThis/window/self, document, require/process, Worker, Reflect, dynamic import, import.meta, `with`, string timers, imports outside the allow list, and reaching `constructor`, `__proto__`, `getPrototypeOf` and similar. Every computed key is rewritten through a runtime guard, so `o["con" + "structor"]` throws. Each refusal carries line, column and reason. This is the static layer; the second layer (running the result in a separate realm without network) is still to build.

2.12 Desktop host and executables

Tauri 2 shell (`src-tauri/`) showing the UI in a native window. Commands: `run_wasm`, `host_info`, and the guest runner `guest_start / guest_resume / guest_swap / guest_end`: the page lifts and lowers a program; Rust instantiates the import-free module in wasmtime, answers system calls with the same shim as the page (guest.rs, kept in step with linux.ts), hands out guest fuel in slices of 5 million dispatches, and returns `needs-code` when the program reaches code not lifted yet, so the page lifts more and swaps the module while memory and CPU state carry over. State layout: registers at 0x0 (72 slots), pc 0x300, flags 0x308 (Z S C O K P), fuel 0x320, code 0x328, fault 0x330.

Executables: `bin/forge` (Node 22+) runs `dist/forge.mjs`; `npm run cli:standalone` builds `dist/forge-linux-x64`, one file with Node inside, without network access (scripts/sea-inject.mjs writes the blob as an ELF note and flips the fuse); `scripts/make-sea.sh` does the same on macOS and Windows with postject. `npm run desktop:build` builds installers: `vite build --mode desktop` produces a static single-page app (no server, no hosted-preview chrome) that scripts/desktop-dist.mjs copies to src-tauri/frontend.

2.13 Pooled model inference (new)

Each compute device holds a contiguous slice of layers sized to its memory; the planner gives faster devices more layers but never more than fits. Activations travel between devices over the link (HTTP in the tests); weights never do: each device builds or reads only its slice. If a device drops out mid-answer, the host re-plans over who is left and redoes that token; if what is left cannot hold the model, the request fails with the numbers ("model needs 1.9 MB; the 1 online device has 1.3 MB"). Tested with separate processes: a model too big for either of two devices answers identically to a single machine, fails cleanly when one is unplugged, and recovers when a third joins. The model is a deterministic toy; a real runtime plugs in at `runLayers`.

2.14 Multiplayer, devices, native mode

Unchanged from the first handoff and still accurate: device shell levels 0 to 4, one owner identity with device certificates and pairing codes, host-authoritative replication with visibility, detail, budget and deltas, zones and ghosts across the owner's machines, visitors who relay and seed but never decide, native mode in two stages (appliance, then bare-metal Rust with a Linux compatibility layer and driver shim). Replication is now carried by the island node's `/island/sync`; prediction, zones across machines and local mesh remain planned.

2.15 Verifying the handoff

```
scripts/offline-bundle.sh fetch     # once, on a machine with internet (vendors crates,
caches npm)
scripts/offline-bundle.sh install   # then anywhere, offline: install, test, build the CLI
npm run test:engine            # 107 tests: engine, x86/ARM vs real-CPU fixtures, glibc,
islands, skills, scripts, pooling
rustc --edition 2021 --test src-tauri/src/guest.rs && ./guest   # the Rust shim, no crates
needed
bin/forge run fixtures/linux/libc-float-x86_64.elf               # a glibc program with
floating point
npm run cli:standalone         # dist/forge-linux-x64, a single file
npm run desktop:build          # the desktop app installer
npm run forge:run -- fixtures/linux/workload-aarch64.elf    # an ARM program running here
sh fixtures/linux/build.sh     # optional: rebuild fixtures (gcc, clang with aarch64,
ld.lld; x86-64 host)
npm run forge:corpus -- 500 7  # optional: fresh random corpus from the hardware oracle
cd src-tauri && cargo test     # first Rust build
npm run dev                    # web UI on :8080 (drop a Linux program on the window)
cd src-tauri && cargo tauri dev   # same UI, native window
```

3. Wants and needs for the next model

Work in this order. Each item names the test that proves it is done. Keep every existing test passing.

3.1 Ordered work list

#	Task	Done when	State now
1	First native build. Compile src-tauri, fix what the toolchain reports, pin the wasmtime version.	cargo test passes; the app opens a window showing the world on one x86 and one ARM machine.	**Ready to try: scripts/offline-bundle.sh, icons and frontend build in place. Needs crates.io once.**
2	Hand-test movement. Third person, first person and bench in the native window and a browser.	Avatar visible in third person; A/D turn; first person looks and strafes; bench orbits freely.	Open. Needs a browser with npm installed.
3	Production frontend for the shell (static build or Node sidecar).	cargo tauri build produces an installer that runs offline.	Open
4	Portals in the world.	Two machines on one network: walk through, see the other island's manifest name and grants.	**Built and tested on one machine (engine, node, UI hook). Needs the two-machine hand test.**

5	Island content transfer.	Visitor renders the host's scene; sha256 of the entry matches the manifest.	**Done (tested over HTTP)**
6	Grow the compat IR: loads and stores, compare and branch, call and return, flags.	Random-program tests across x86, ARM, WebAssembly agree with the stepper.	**Done, with the real CPU as the oracle**
7	Skill registry.	A cloud-written skill is accepted only after its fixture passes, then runs offline.	**Done**
8	Safe-compile gate for scripts.	A script with a forbidden call is rejected with the reason.	**Done (static layer)**
9	Adjacency: two opted-in islands side by side, live.	Two players see each other's islands live; stopping one host removes its half.	**Open. island-content.ts sync is the building block.**
10	Native binary route.	A small Linux program runs inside Forge.	**Done for static x86-64 and ARM64 programs, and static glibc programs on x86-64**
11	Shell levels 1 and 2 on Android.	A tester uses Forge as home screen for a week and can switch back in one step.	**Open**
12	Linked devices over the network.	Phone drives the PC's world; turn off the PC and the phone keeps it running.	**Open**
13	Replication on a transport, plus client-side prediction.	Two visitors, each under budget; walking feels instant at 150 ms added latency.	**Transport, budgets and deltas done and tested. Prediction open.**
14	Zones on several machines with live authority handoff.	A border crossing between two machines with no visible hitch.	**Open**
15	Local mesh.	Three phones with no internet share one world; a fourth downloads faster from peers.	**Open**
16	Pooled inference.	A model too large for any one of two linked machines answers using both; unplugging one re-plans or fails cleanly.	**Mechanics done and tested across processes with a toy model. Real GGUF per-layer runtime open.**
17-19	Native stage A appliance; Linux compatibility layer in the Rust core; native stage B bare metal.	As in the first handoff.	**Open. 2.8's shim is a starting point for 18.**

Items N1 to N3 (from the second edition)

- **N1. Guest programs in the native host:** written and tested in parts. Done when the workload and glibc fixtures pass through wasmtime: `cargo test` plus one run of `npm run desktop:dev` dropping a fixture on the window.
- **N2. glibc static binaries:** done for x86-64 (printf, malloc, qsort, strings, floating point). ARM64 glibc is next: NEON loads and compares, tpidr_el0, exclusives.
- **N3. Second layer for scripts:** done (worker, message-only API).
- **N4. Real model slices:** per-layer forward for a GGUF model behind `SliceLink`. Open.

Next, in order

- **N5. Compile and ship the desktop app** on one x86 and one ARM machine (work item 1 plus N1's acceptance test).
- **N6. ARM64 glibc:** extend gen-compat-corpus.ts with NEON, exclusives and tpidr_el0; an ARM64 machine as a second oracle would let the corpus check ARM directly.
- **N7. Files for guests:** a per-island virtual file system behind the shim (open/read/write/close on a capability-scoped tree), so command-line tools can work on files.
- **N8. Packed SSE float and AVX** for compilers' vectorized loops (today those programs stop at a reported gap).

3.2 Guardrails the next model must keep

- Never execute bytes the engine cannot verify. Gaps are reported, not guessed.
- New opcodes enter the decoders only with a test whose expected answer came from real hardware (fixtures/linux or the corpus), not from the decoder's own idea of the semantics.
- The CPU is the oracle. Never read flags the architecture leaves undefined in generated tests (bsf/bsr, multiply ZF/SF, OF after multi-bit shifts); they differ between processors.
- Guest WebAssembly modules import nothing. Everything a guest program can do goes through the shim, and the shim only grants what is listed in 2.8.
- Never widen a grant. Hosts grant a subset of the request; the visitor checks that.
- Never show island content whose hash does not match the host's signed manifest.
- Never let a model write directly into the running engine. Proposals go through a fixture or a schema check; a picker's output is always validated.
- Keep the private key private. Do not log it, send it, or put it in a manifest.
- Keep the movement code single-owner: the pilot owns the camera; nothing else writes it each frame. (This pass added portals and drop-to-ingest without touching the pilot.)
- Do not reintroduce the software overlay as a default. It caused the blur and the crude avatar.
- Do not reverse-engineer closed commercial software. Open-source and the owner's own code only.
- Every device takeover step must have a tested way back to the stock system. Never lock an owner out of their own phone, emergency calling included.
- Only devices the owner certified may join an instance. A removed device must stay removed.
- Authority stays on machines the island owner controls. Visitors relay, seed and render; they never decide what happens.

3.3 Open decisions for the owner

- **Transport.** Plain HTTP works on a home network. Across the internet, hosts behind home routers need WebRTC with a signaling service, a relay, or port forwarding. Session tokens need HTTPS outside a home network.
- **Discovery.** How people find islands: a shared directory, friends' portal lists, or a peer-to-peer index.
- **Persistence.** Whether popular islands may be cached by peers after the owner goes offline.
- **Models.** Which small model routes intent locally, and which cloud model writes skills.
- **Licensing.** Open-source license for Forge itself, and rules for third-party island content and skills.
- **Lead with AI?** Whether pooled inference ships before islands, as the first reason to install.
- **First native hardware.** Which machine family stage B targets first.
- **Which programs first.** Static musl command-line tools are the nearest win; glibc, Windows (PE) and GUI programs are each a large step.

3.4 What the next model needs

- This repository, plus a machine with Rust, Node 22+ and the Tauri system libraries.
- Network access to crates.io and npm.
- For new opcodes: gcc, clang with the aarch64 target, ld.lld, and an x86-64 machine (the hardware oracle). An ARM64 machine would let the corpus check ARM directly too.
- Two devices (ideally one x86 and one ARM) on the same network for portal and pooling tests.
- The owner's answers to the open decisions above, starting with transport.

3.5 Known limits, stated plainly

- Perfect translation of arbitrary programs between x86 and ARM is not solved by anyone. Forge runs what its map covers, refuses the rest safely, and grows the map with hardware-checked tests. Today that map is the integer core of compiled code.
- The x86 flags after shifts and multiplies are approximated (zero and sign are exact; carry and overflow are not modelled). Compilers rarely branch on them; the random corpus avoids them; a program that does could differ.
- Closed, server-backed apps can only be wrapped through their public interfaces.
- Seamless worlds spanning untrusted machines are harder than in one company's data center, because no host can trust another's simulation. Ownership per entity is the starting rule.
- Static analysis of JavaScript can be argued with; the script gate needs its runtime second layer before it is trusted with hostile code.
- Bare-metal boot needs a separate native core and is the last step, not the first. Bare metal makes the system leaner and steadier, not raw computation much faster.
- Each platform decides how far an app can take over. iOS allows the least; Android with an unlocked bootloader and Linux allow the most.